DOCUMENT 01: DEVOPS DEPLOYMENT GUIDE

System Runbook for Continuous Integration & Cloud Infrastructure Provisioning

1. Initialization & Version Control Workflow

The production architecture sequence begins with initializing an isolated local environment wrapper and standardizing repository pipelines using Git source control structures. Execute the deployment sequence lines inside the local workstation terminal layer:

```
git init git add . git commit -m "Production Build v2.5.0: Integrated Dynamic Relational SQL & AWS Ingestion Matrices" git branch -M main git remote add origin https://github.com/Lalith-Cloud-Architect/leo-cloud-gateway.git git push -u origin main
```

2. AWS Cloud Infrastructure Setup (IAM & S3)

To ensure secure communication across the service gateways without exposing master accounts, the following infrastructure deployment protocol is required:

1. Navigate to the **AWS Management Console** pointing target nodes to the Stockholm `eu-north-1` cloud region cluster.
2. Access the **IAM (Identity and Access Management) Dashboard** and create an isolated programmatically accessible deployment user.
3. Generate a secure credentials token pair consisting of an `AWS_ACCESS_KEY_ID` and an `AWS_SECRET_ACCESS_KEY`.
4. Navigate to the **S3 Console** and create a secure data object storage block bucket explicitly assigned the unique globally bounded string namespace identifier: `leo-optimized-bucket-lalith`.

3. Render Platform Integration Sequence

To transition the local microservices to a production live runtime framework, execute the cloud provisioning lifecycle mapping sequence within the Render platform network nodes:

- Log in to the Render Console and link your validated GitHub source repository.
- Provision a new **Web Service** container using the Python 3 native environment architecture stack.

- Specify the required production runtime server invocation trigger command string sequence inside the platform console parameters layer: `uvicorn leo_core_gateway:app --host 0.0.0.0 --port $PORT`.

DOCUMENT 02: CYBERSECURITY & IDENTITY COMPLIANCE REPORT

Security Architecture Matrix, IAM Principle of Least Privilege, and Token Masking Definitions

1. Privilege Mitigation Strategy (IAM Policy Document)

To maximize cloud perimeter protection, the system rejects unrestricted administrative access. The application configuration utilizes the security standard **Principle of Least Privilege (PoLP)**. The programmatic worker token is restricted to executing data ingestion and read-loop queries strictly inside the LEO storage grid using the following optimized JSON security schema policy matrix:

```
{ "Version": "2012-10-17", "Statement": [ { "Sid":
"LeoS3StorageIngestionLeastPrivilege", "Effect": "Allow", "Action": [
"s3:PutObject", "s3:GetObject", "s3:ListBucket" ], "Resource": [
"arn:aws:s3:::leo-optimized-bucket-lalith", "arn:aws:s3:::leo-optimized-bucket-
lalith/*" ] } ] }
```

2. Data Defenses & Threat Modeling Matrix

The framework integrates dedicated DevSecOps validation mechanisms to manage and isolate transactional traffic variables safely across production gateways:

- **Credential Isolation Masking:** Sensitive access profiles are strictly banned from code-level persistence. API components ingest configurations dynamically using secure `os.getenv` runtime hooks bound to encrypted variables.
- **Cross-Site Scripting (XSS) Mitigation:** A global Cross-Origin Resource Sharing (CORS) security header enforcement block intercepts client transactions, allowing only signed method types.
- **Data Injection Guardrails:** Database interactions avoid arbitrary raw string formatting, employing parameterized, bound array queries via SQLAlchemy and sqlite3 interfaces to block malicious payload variants.

DOCUMENT 03: API REFERENCE MANUAL

Core REST Endpoint Definitions, Schema Assertions, and Transaction Framework Mappings

1. Active Architectural Endpoint Configurations

The system gateway registers and listens on specialized routes designed to handle processing matrix parameters and telemetry streams:

Route Method	Endpoint Destination	Payload Interface Requirement	Transactional Processing Output Effect
GET	`/interface`	None (Browser Context Open)	Renders the full graphical HTML/Tailwind web console management dashboard page.
POST	`/office/submit-comment`	JSON Model: `OfficeCommentPayload`	Performs dual-action data injection into local SQL log tables and live AWS S3 storage streams.
POST	`/office/submit-rating`	JSON Model: `RatingPayload`	Commits performance score indicators directly into the primary relational feedback structures.
GET	`/office/rating-stats`	None	Queries tables to extract, compute, and return real-time rating telemetry averages.

2. Schema Schema Payload Specifications

2.1 Office Log Ingestion Ingestion Template (POST Request Body)

```
{ "user_name": "string (Required: Unique System Cloud Identifier Key)",
"comment_text": "string (Required: Raw Uncompressed Transaction Log Chunks)" }
```

2.2 Response Model Matrix (200 OK Status)

```
{ "status": "SUCCESS", "mode": "LIVE_AWS_PRODUCTION / SIMULATION", "target_asset":
"s3://leo-optimized-bucket-lalith/office-comments/comment_timestamp.txt",
"security_audit": "INTEGRITY_CHECK_PASSED (SHA-256 Token Mask)" }
```

DOCUMENT 04: USER ACCEPTANCE TESTING (UAT) LOG

System Assurance Matrix, Component Quality Controls, and Integration Assertions

1. Production Assurance Execution Logs

The validation sequence evaluated the framework's behavior under edge conditions and high-throughput operational runs. The verified behavioral test logging results are captured below:

Test ID	Target Component Check	Simulated Operational State Input Trigger	Expected System Processing Behaviour Output	Status
UAT-001	Input Data Validation Handling	Execute submit payload with blank spaces or null characters.	Intercept operation and return `[VALIDATION_ERROR]` string without crashing pipelines.	PASSED
UAT-002	DevSecOps Connection Fallback	Wipe AWS token credentials from the active env variables tracking map.	Gracefully shift system states into `SIMULATION MODE` to keep the UI active.	PASSED
UAT-003	Dual-Action Data Ingestion Sync	Trigger text array submission block on the interface terminal.	Simultaneously write records to the local SQL audit rows and generate real S3 file objects.	PASSED
UAT-004	Dynamic Rating Aggregation Engine	Submit multiple 4-star and 5-star evaluation scores via the dashboard widget.	Execute real-time mathematical score aggregation averages and refresh display interfaces.	PASSED

2. Verification Architecture Approval

All functional validation passes completed with zero service failures, runtime performance blocks, or exception leaks across the gateway interfaces. The system runtime has successfully satisfied the verification controls.

```
PROJECT ASSURANCE MASTER LOG SHIELD IDENTIFICATION: LEO_V2.5.0_COMPLIANCE_APPROVED
```